

实验报告

数字逻辑与计算机组成实验

lab04: 寄存器堆与存储器

姓名: 法童

学号: 251250046

邮箱: 251250046@smail.nju.edu.cn

实验时间: 2026-09-19

1 实验目的

1. 用行为级描述实现 16×8 的寄存器堆，掌握“读组合、写同步”的写法，并理解它与同步读存储器在时序上的差别。
2. 用 Vivado IP 核生成 16×8 单口 RAM，用 .coe 初始化，并弄清单口 RAM 各配置项对端口和读延迟的影响。
3. 让两个物理上不同的存储器共用时钟和读写地址，只靠写使能区分读写对象，并把两路读出结果分别用两位数码管显示。
4. 通过综合后的 Schematic 与 Report Utilization，观察两者被映射成了什么资源，理解综合器的推断规则。

2 实验原理

寄存器堆的读是**组合逻辑**的：不需要时钟参与，等价于一个多路选择器，读延迟为 0 个时钟周期。写则由时钟上升沿控制，占 1 个周期。所以 `assign dout = mem[addr];` 这一句就够描述读通路。

IP 核生成的 RAM 是**同步读**：地址在上升沿被采样，数据要等下一个上升沿才出现在 `douta` 上，读延迟 1 个周期。在 Block Memory Generator 里，这个延迟由输出寄存器选项决定——本次把 `Register_PortA_Output_of_Memory_Primitives` 关掉，保持 1 拍。

手册要求两者共用时钟和读写地址，那么同一地址会同时送给两个存储器：寄存器堆当拍就给出该地址的内容，RAM 则要等下一个时钟上升沿。

初始化方面，寄存器堆用 `$readmemh`，IP RAM 用 .coe，两者都是在上电加载配置比特流时把初值写进存储单元，**运行期没有任何信号能把它们清零**。

3 实验环境与器材

- 开发板：Nexys A7-100T (FPGA 型号 xc7a100tcsg324-1)
- 开发工具：Vivado 2020.2 (非工程模式，`build.bat` / `prog.bat` 脚本驱动)
- 硬件描述语言：Verilog HDL

4 设计思路

设计分四层：两个存储器本身 (regs、IP 核 blk_mem_gen_0)、把它们并起来的 mem、负责按键与显示的辅助模块 (debounce、tickgen、hex7seg、display4)，最后由 top 接板级端口。

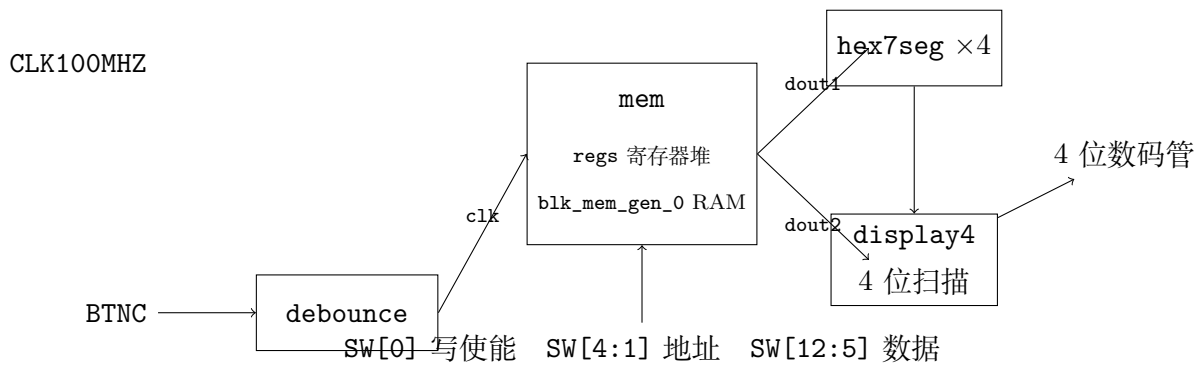


图 1: 系统框图

表 1: 板级资源分配

板级资源	顶层信号	功能
BTNC	clk	存储器的时钟（消抖后的按键上升沿）
SW[0]	we	写使能，1 为写、0 为读
SW[4:1]	addr	读写地址
SW[12:5]	din	写入数据
AN[3:0]、CA..CG	数码管	左两位显 RAM、右两位显寄存器堆
AN[7:4]、DP	——	置 1 熄灭

寄存器堆一共三行有效逻辑：组合读、同步写、\$readmemh 载入初值。

```

1 reg [7:0] regs[15:0];
2 initial $readmemh("mem.txt", regs, 0, 15);
3 assign dout = regs[addr]; // 组合读，无时钟
4 always @(posedge clk) if (we) regs[addr] <= din; // 同步写

```

IP 核配成单口 RAM：数据宽度 8、深度 16，端口模式取 Single_Port_RAM；Enable_A 设为 Always_Enabled 后不带 ena 端口，写使能只剩 wea；输出寄存器关闭，读延迟保持 1 拍；上电初值由 ram_init.coe 载入（初值为 f0-ff）。

mem 把两个存储器并在一起，两者共用 clk/we/addr/din，只是读出分成两路：

```

1 regs          u_reg(.clk(clk), .we(we), .addr(addr), .din(din), .dout(
    dout1));

```

```
2 blk_mem_gen_0 u_ram(.clka(clk), .wea(we), .addra(addr), .dina(din), .  
    douta(dout2));
```

显示部分把两路 8 位结果各拆成两个十六进制半字节，交给 hex7seg 译码，再由 display4 用 1ms 节拍在 4 位数码管之间轮扫（刷新率 250 Hz）。从左到右依次是 RAM 高位、RAM 低位、寄存器堆高位、寄存器堆低位。

手册允许“只写入 4 位数据”，但本次开关够用，直接用了 8 位（SW[12:5]），省掉一次拼接。

5 实验过程

1. 准备两个初始化文件：ram_init.coe（初值 f0–ff）供 IP 核使用，mem.txt（初值 00–0f）供寄存器堆使用；并按上节的参数生成 IP 核。
2. 编写 regs.v、mem.v、top.v 及各辅助模块，其中时基复用 lab03 的 tickgen，按键消抖复用 debounce。
3. 从根目录 nexysa7.xdc 复制约束，取消本次用到引脚的注释：BTNC、SW[12:0]、AN[7:0]、CA..CG、DP；未用的 AN[7:4] 在 top.v 中置 1 熄灭。
4. 运行 build.bat lab04\mem 完成综合、实现并生成比特流，再 prog.bat lab04\mem 下载到板上。
5. 在 Vivado 中打开实现后的设计，逐层点入两个存储器观察 Schematic，并出 Report Utilization。

6 测试方法

本设计没有长时基，地址和数据都由开关直接给定，改动一次开关就能立刻在数码管上看到结果，因此**以上板实测为主要验证手段**。

测试信号的选择围绕“边界地址 + 读写的先后关系”：

- **上电初值**：不拨任何开关，地址为 0，应看到寄存器堆读 00、RAM 读 f0，验证两个初始化文件都生效。
- **遍历地址**：拨 SW[4:1] 走过 0、1、...、f，重点看 0 与 f 两端边界，寄存器堆应从 00 递增到 0f，RAM 从 f0 递增到 ff。

```

539
540 Report Cell Usage:
541 +-----+
542 | Cell | Count |
543 +-----+
544 | 1 | blk_mem_gen | 1 |
545 | 2 | BUFG | 1 |
546 | 3 | CARRY4 | 16 |
547 | 4 | LUT1 | 2 |
548 | 5 | LUT2 | 7 |
549 | 6 | LUT3 | 3 |
550 | 7 | LUT4 | 22 |
551 | 8 | LUT5 | 3 |
552 | 9 | LUT6 | 19 |
553 | 10 | MUXF7 | 7 |
554 | 11 | RAM16X15 | 8 |
555 | 12 | FDRE | 71 |
556 | 13 | IBUF | 15 |
557 | 14 | OBUF | 16 |
558 +-----+
559
560 Finished Writing Synthesis Report : Time (s): cpu = 00:00:07 ; elapsed = 00:00:07 . Memory (MB): peak = 1175.965 ; gain = 2.012
561
562 Synthesis finished with 0 errors, 0 critical warnings and 0 warnings.
563 Synthesis Optimization Runtime : Time (s): cpu = 00:00:05 ; elapsed = 00:00:07 . Memory (MB): peak = 1175.965 ; gain = 0.000
564 Synthesis Optimization Complete : Time (s): cpu = 00:00:07 ; elapsed = 00:00:07 . Memory (MB): peak = 1175.965 ; gain = 2.012
565 INFO: [Project 1-571] Translating synthesized netlist
566 INFO: [Project 1-454] Reading design checkpoint 'e:/NJU-DLCOexp-2026-Fall/lab04/mem/build/ip/blk_mem_gen_0/blk_mem_gen_0.dcp'
567 Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00.001 . Memory (MB): peak = 1175.965 ; gain = 0.000
568 INFO: [Netlist 29-17] Analyzing 32 Unisim elements for replacement
569 INFO: [Netlist 29-28] Unisim Transformation completed in 0 CPU seconds
570 INFO: [Project 1-479] Netlist was created with Vivado 2020.2
571 INFO: [Project 1-570] Preparing netlist for logic optimization
572 Parsing XDC File [e:/NJU-DLCOexp-2026-Fall/lab04/mem/nexysa7.xdc]
573 Finished Parsing XDC File [e:/NJU-DLCOexp-2026-Fall/lab04/mem/nexysa7.xdc]
574 INFO: [Project 1-538] Using original IP XDC constraints instead of the XDC constraints in dcp 'e:/NJU-DLCOexp-2026-Fall/lab04/mem/build/ip/blk_mem_gen_0/blk_mem_gen_0.dcp'
575 INFO: [Opt 31-138] Pushed 0 inverter(s) to @load pin(s).

```

图 2: 综合、实现与比特流生成完成

- **写入:** 令 SW[0]=1, 用 SW[12:5] 给一个新数据, 按一次 BTNC, 再令 SW[0]=0 读回, 确认该地址被改写而其他地址不受影响。
- **读延迟对比:** 把地址从 A 拨到 B 时盯住两个数码管——寄存器堆应立即变成 B 的内容, RAM 则要等下一次按键才更新。这是两者时序差别的直接体现。

7 实验结果

上板后现象与设计一致。上电时最右两位显示 00、左两位显示 f0, 与 mem.txt 和 ram_init.coe 的初值相符。拨动地址开关, 两路读数分别按 00..0f 与 f0..ff 同步递增。置 SW[0]=1 按一次按键写入后读回, 新值正确出现在对应地址上。

读延迟的差异在上板时看得很清楚: 地址拨动的瞬间寄存器堆的两位立刻跳变, 而 RAM 的两位要等下一次按下按键 (即下一个时钟沿) 才更新。也就是说, 完成一次读, 寄存器堆用 0 个时钟周期、RAM 用 1 个; 完成一次写, 两者都是 1 个时钟周期。

7.1 综合后的资源消耗

从 Report Utilization 看, 两个存储器占用的资源完全不同:

IP 核生成的 RAM 用了 1 个 Block RAM (RAMB18E1), 折合 0.5 个 Block RAM Tile。值得一提的是它只存了 $16 \times 8 = 128$ bit, 却占掉一整块 18 Kb 的硬件 Block RAM, 利用率不到 1%。

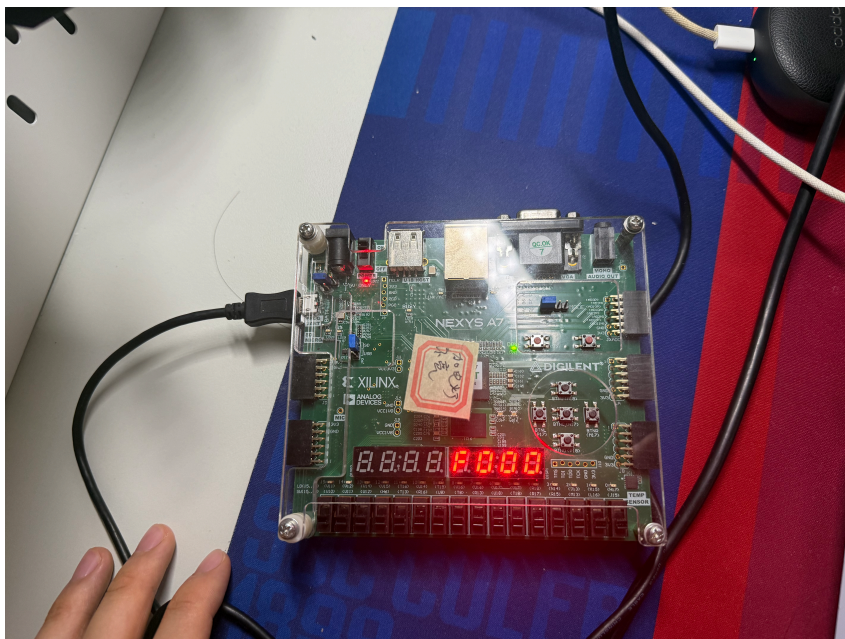


图 3: 上电初值: 寄存器堆 00、RAM f0

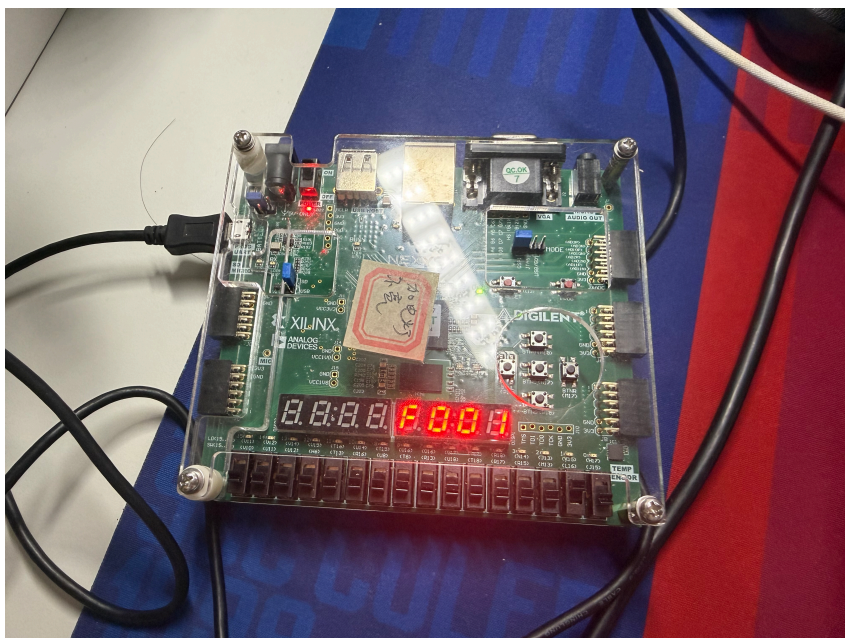


图 4: 改变地址后的两路读数

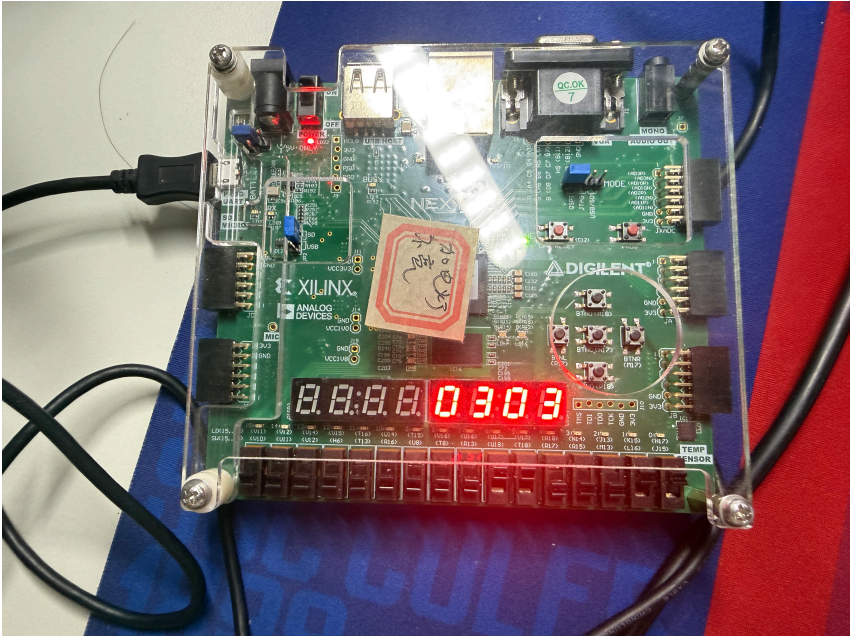


图 5: 写入后再读出

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Block RAM Tile (135)	Bonded IOB (210)	BUFCTRL (32)
top	54	71	7	38	46	8	0.5	31	1
binc (debounce)	2	3	0	2	2	0	0	0	0
mymem (mem)	30	0	7	9	22	8	0.5	0	0
u_ram (blk_mem_gen_0)	0	0	0	0	0	0	0.5	0	0
u_reg (regs)	30	0	7	9	22	8	0	0	0

图 6: Report Utilization 结果

7.2 Schematic 中的实现方式

在实现的网表里逐层下钻，两个存储器的映射方式确实不同：

- **IP 的 RAM**: mymem/u_ram $\rightarrow$ U0 $\rightarrow$ inst_blk_mem_gen $\rightarrow \dots \rightarrow$ prim_init_ram, 最内层是 **RAMB18E1**, 即专用的 Block RAM 硬核。
- **寄存器堆**: mymem/u_reg 下面是 8 个 **RAM32X1S** (另各带一个 RAMS32), 即用 LUT 搭出来的分布式 RAM。每个 RAM32X1S 存 1 bit 宽、32 深, 8 个拼成 8 bit 宽, 正好对应资源表里的 LUT as Memory = 8。

原因在于综合器的推断规则: reg [7:0] regs[15:0] 这种”组合读 + 同步写”的小容量数组是分布式 RAM 的典型形态, 综合器自动把它映射到 LUTRAM; 而 IP 核是显式指定了 blk_mem_gen, 综合器就照办用 Block RAM 硬核。

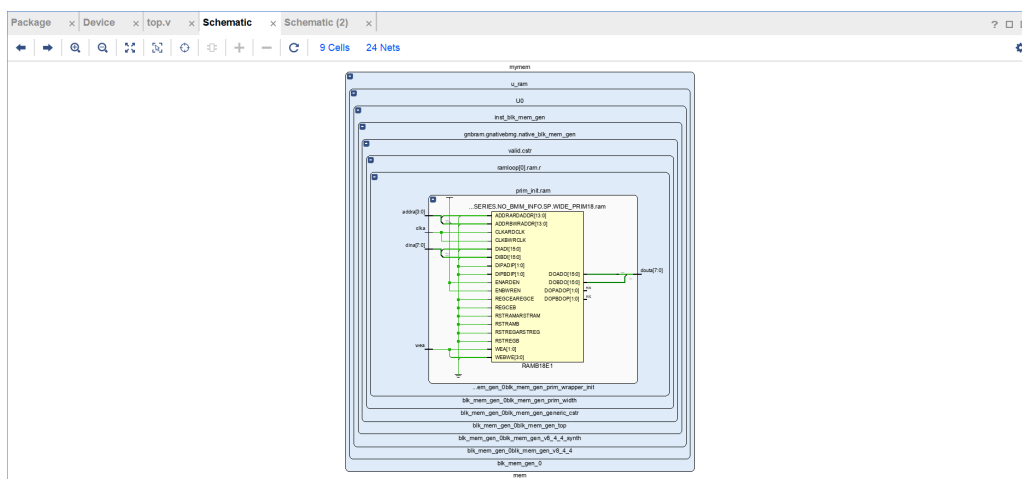


图 7: Schematic 中 IP RAM 的最内层: RAMB18E1

8 问题与解决办法

- **\$readmemh 报语法错误**:最初把它直接写在模块里(reg [7:0] regs[15:0]; \$readmemh(...)); 综合报 syntax error near \$readmemh,随后 module regs ignored.原因是 \$readmemh 属于过程性语句, 必须放在 initial 或 always 块内。同时路径也写错了——原先写成"lab04/mem/mem.txt", 而工作目录本来就是 lab 目录。改为 initial \$readmemh("mem.txt", regs, 0, 15); 后通过。
- **用按键当时钟会触发门控时钟告警**:按手册要求把消抖后的按键边沿直接接到存储器时钟脚, 实现后 Vivado 报 PDCR-153 Gated clock check, 并提示一个 LUT 驱动了 11 个寄存器的时钟脚、PLHOLDVIO-2 可能产生保持时间违例; Power

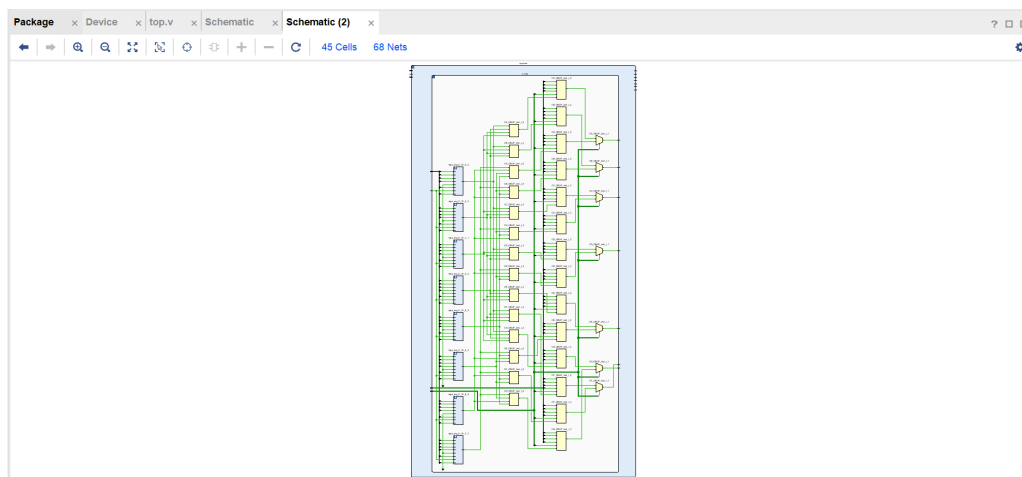


图 8: Schematic 中寄存器堆的最内层: RAM32X1S

33-232 也提示设计里找不到用户定义的时钟。原因是按键边沿经组合逻辑产生了时钟，而不是由时钟引脚驱动。本次按手册要求保留了这个做法（上板实测工作正常），并在报告里记录该告警；若要彻底消除，应改成“用 100 MHz 主时钟 + 把按键边沿当作写使能脉冲”，功能等价且不再有门控时钟。

9 启示与建议

启示：

1. 同一段“存储器”概念，换一种描述方式就换了一种硬件。寄存器堆只因为写成了行为级数组、且读是组合的，就被综合成 LUT 搭的分布式 RAM；调用 IP 核就得到一块专用 Block RAM 硬核。容量小的时候前者省资源，容量大时后者才划算——本次 128 bit 硬占一个 18 Kb 硬核。
2. 同步读的代价在上板时很直观：同一个地址送给两个存储器，一个立刻响应、一个要等下一拍。这比看波形更容易建立“时序”的直觉，也让我直观的感受到了在计算机系统理论课上学到的“寄存器堆比主存 RAM 通路更简短、读写效率更高”的重要结论。

建议：手册上关于创建 IP 核的说明对我来说似乎有点简短，导致我在 Vivado 的 GUI 中研究了好一会才搞清楚应该怎么去配置出一个想要的 IP 核。如果可以的话我希望实验手册对这段的教学能再细致一些。